

Architectural Blueprint for Remote Mobile-to-macOS Software Development Orchestration

The paradigm of software engineering is undergoing a critical transition, shifting from direct, manual text editing to high-level systemic oversight managed by autonomous agentic workflows.¹ For developers seeking maximum operational velocity, the ability to safely orchestrate codebase modifications, execute local test suites, and deploy projects to remote production-ready environments from a mobile device represents a significant leap in productivity.³ To realize this workflow without exposing local macOS environments—such as an Apple Silicon MacBook or Mac Mini—to severe security vulnerabilities, developers require a unified, multi-tiered architecture. This system must bridge a lightweight mobile frontend with a secure peer-to-peer network tunnel, an intelligent intent-classifying orchestrator, and a modular execution layer optimized for localized large language models (LLMs).⁵

Orchestrator Selection and Comparative Framework

The central component of the host runtime is the orchestrator, which processes incoming requests from the mobile application, classifies user intent, searches for matching capabilities, and loads the corresponding system workflows. Rather than constructing an orchestration engine entirely from scratch, developers can leverage and adapt mature, open-source agent frameworks to process multi-turn code generation and execution loops.⁷

Framework	Core Intent Classification Mechanism	Local LLM and Tool Integration	Communication Protocol	Context and Memory Management
Dynamic Agent	Two-tier semantic parsing via local JSON schemas and topic inference ⁷	Native integration with Ollama; automatic vector schema detection via vector-nexus ⁷	Real-time WebSocket layer for high-frequency streaming ⁷	Hierarchical caching using Redis (Tier 1) and Qdrant Semantic Cache (Tier 2) ⁷
Open Interpreter	Direct system message	Compatible with any local	Standard CLI standard	Native conversation

	prompt injection and terminal command generation ⁸	OpenAI-compatible API base (Ollama, LM Studio) ⁸	input/output stream pipes ⁸	state array with explicit session clear and resume options ⁸
Agent Squad	Dynamic classifier combining conversation history and specialized agent profiles ⁹	Multi-model compatibility (Anthropic, Bedrock, OpenAI) ⁹	Standard HTTP JSON REST endpoints ⁹	Decentralized conversation message storage and dynamic agent delegation ⁹

To implement an optimal backend runtime on macOS, a hybrid approach combining the strengths of these frameworks is highly effective. Developers can configure a modified variant of the Rust-based **Dynamic Agent** as the primary gateway, leveraging its low-latency WebSocket communication layer and Redis exact-match caching to process incoming mobile inputs instantaneously.⁷ For the execution loops, the system can delegate tasks to an **Open Interpreter** backend running locally, allowing the LLM to run Python, JavaScript, and Bash commands directly within a sandboxed directory while requiring manual verification for destructive operations.⁸

This orchestrator is powered by a local, mid-sized model (such as Qwen-3-Coder-7B or Llama-3.1-8B) deployed via Ollama or Apple’s MLX-LM framework to perform semantic routing and lightweight parsing entirely offline.⁵ This ensures that sensitive project data remains isolated on local hardware.⁵

Specialised Skill Integration Layer

As illustrated in the system architecture, the local orchestrator routes classified user requests to four specialized, modular "Skills" designed to interface directly with the laptop’s file systems, local compilation tools, and diagnostic systems.

Integration Skill	Core Purpose and Capabilities	Primary File Paths and Workspace Resources	Primary Tooling and Executables	Local/Cloud Execution Dependency
Skill 1: Claude	Executes multi-file	Workspace root	Claude Code CLI, native Git,	Cloud Hybrid (Thin local Go

Code	project refactors, traces code dependencies, runs test suites, and manages git commits. ¹	directories, repository-level files, and .claude/settings. ¹⁴	and local compilers (npm, cargo, pip). ³	client with Anthropic API cloud inference). ¹⁵
Skill 2: Codex	Generates, completes, and audits localized blocks of code offline with zero token charges. ¹³	Active buffer files, single-module directories, and custom code templates. ¹³	Ollama / MLX-LM local APIs serving CodeLlama or DeepSeek-Coder. ⁵	Fully Local (Executed entirely on local Apple Silicon GPUs). ⁵
Skill 3: Antigravity	Coordinates parallel subagents, manages task checklists, and tracks implementation histories. ²	Brain task directories located under ~/.gemini/antigravity/brain/<GUID>/. ¹⁸	Google Antigravity CLI, Antigravity SDK, and antigravity-cli.py helper. ¹⁸	Cloud Hybrid (Leverages server-side harnesses optimized for Gemini 3.5). ¹⁶
Skill 4: CPU Activity & Process Manager	Monitors workstation thermals, tracks memory utilization, and terminates resource-intensive background processes. ²⁰	/proc system files, virtual memory logs, and execution thread records. ²⁰	Python interpreter executing script interfaces built on the psutil library. ²⁰	Fully Local (Direct system call execution on the macOS host). ²¹

Implementing Skill 1: Claude Code Command Delegation

To orchestrate Claude Code programmatically, the orchestrator invokes the CLI in non-interactive mode using the `--bare` and `-p` flags.²³ This bypasses the interactive terminal interface and returns raw, structured JSON formatting.²³ To execute unconfirmed test-and-repair cycles safely, the system passes specific tool permissions via the command

string²³:

Bash

```
claude -p "Execute the unit test suite and repair any syntactical failures" --bare --allowedTools  
"Bash,Read,Edit" --output-format json
```

This pipeline allows the orchestrator to capture step-by-step progress events and pass stream-compatible terminal outputs back to the mobile WebSocket interface.²³

Implementing Skill 3: Antigravity Task Manipulation

When coordinating larger projects, Google's Antigravity platform stores task states as structured Markdown checklists and metadata payloads in the host's directory.¹⁸ To integrate this programmatically, the orchestrator utilizes the `antigravity-cli.py` utility.¹⁸ The orchestrator queries the active state of tasks, updates subtask checkboxes, and tracks versioned implementation plans using precise CLI calls:

Bash

```
# Query the ordered task checklist from task.md to locate incomplete blocks  
antigravity-cli.py show-task 1
```

```
# Set a critical task state boundary to execution phase with localized metadata summary  
antigravity-cli.py update-task --primarytask 1 --subtask 3 --state in-progress --summary  
"Initiating remote codebase compilation"
```

```
# Back up and version the modified script in the Antigravity history pool  
antigravity-cli.py write-artifact --artifacttype implementation_plan --primarytask 1  
--filepath./src/main.py --summary "Synchronized codebase from mobile client"
```

This interaction keeps the mobile client updated on task progression and provides clear boundaries for safe agent execution.¹⁸

Implementing Skill 4: System Telemetry via psutil

To monitor host system performance during intensive coding routines, Skill 4 queries operating system performance records using Python's cross-platform `psutil` library.²⁰ The orchestrator

runs a background daemon that serializes system statistics into a structured telemetry payload
21.

Python

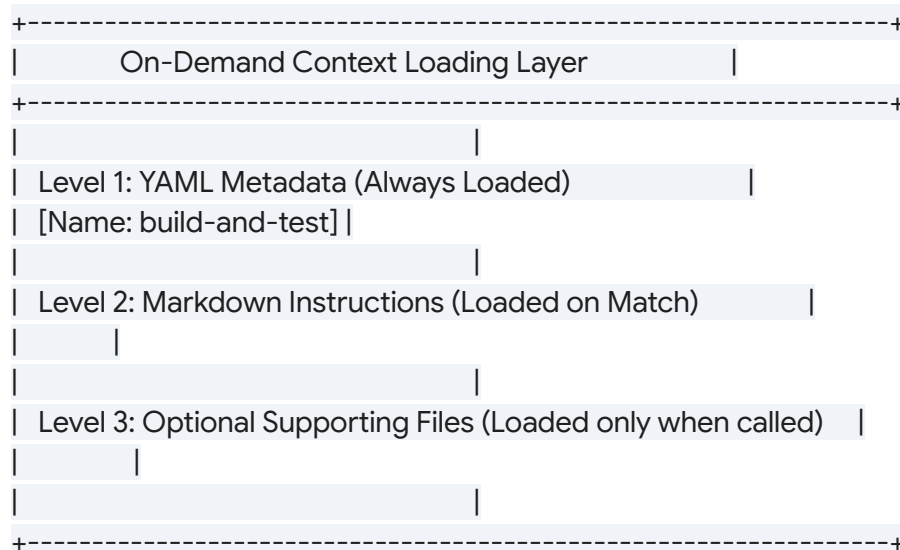
```
import psutil [21]

def retrieve_telemetry_payload() -> dict:
    return {
        "cpu_count_physical": psutil.cpu_count(logical=False)[21]
        "cpu_count_logical": psutil.cpu_count(logical=True)[21]
        "cpu_utilization_aggregate": psutil.cpu_percent(interval=1)[21]
        "memory_percent_used": psutil.virtual_memory().percent[21]
        "active_processes": [
            {
                "pid": proc.info["pid"],
                "name": proc.info["name"],
                "cpu_percent": proc.info["cpu_percent"],
                "memory_percent": proc.info["memory_percent"]
            }
            for proc in psutil.process_iter(["pid", "name", "cpu_percent", "memory_percent"]) [21]
            if proc.info["cpu_percent"] > 5.0 or proc.info["memory_percent"] > 2.0
        ]
    }
```

This prevents the workstation from thermal throttling or running out of memory during concurrent execution loops by exposing real-time process management controls to the mobile client.⁵

Claude Skills Architecture and Local Context Management

To keep the local 7B-9B parameter model highly responsive, the system implements the modular **Claude Skills Architecture**.²⁵ Standard agent architectures degrade in performance when all instructions, tool definitions, and system prompts are loaded into the context window at once.²⁶ The Skills pattern addresses this by loading resources on-demand across three distinct levels.²⁵



Three Levels of Skill Loading

- **Level 1 (YAML Metadata):** During initialization, the orchestrator parses only the top-level YAML frontmatter of each skill, adding name and description configurations to the system prompt.²⁵ The model knows a capability exists but does not consume token space on the full instruction set.²⁵
- **Level 2 (Markdown Instructions):** When the model identifies a matching intent, it triggers the skill loader.²⁶ The system reads SKILL.md from the filesystem and injects the step-by-step instructions into the active conversation context.²⁵
- **Level 3 (On-Demand Supporting Files):** If the instructions in SKILL.md reference local templates or code examples, the model fetches them on-demand only when executing that specific step.²⁵

Scoped Configurations and Context Budgets

To facilitate team workflows and maintain personal preferences, skills are organized into two distinct file directories on the host machine ¹⁴:

```
# Personal Configuration Scope (Private workflows across all user projects)
~/claude/skills/<skill-name>/SKILL.md
```

```
# Project Configuration Scope (Shared repository-specific coding standards)
<project-root>/claude/skills/<skill-name>/SKILL.md
```

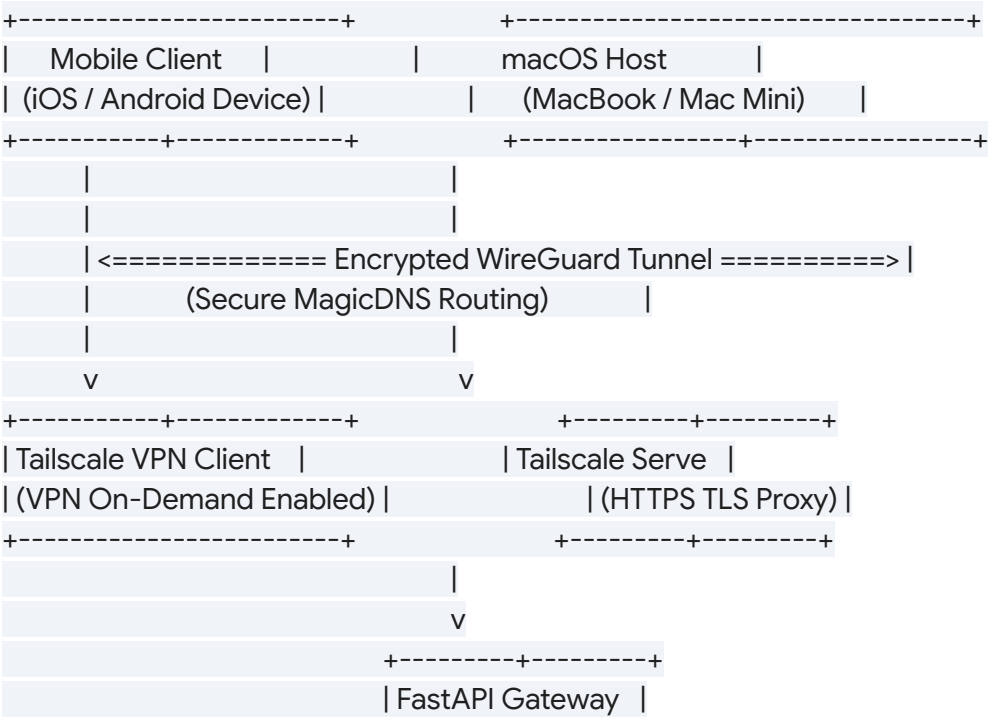
To prevent context window saturation, the orchestrator implements an automatic context-compaction routine.¹⁴ When the conversation chain approaches the model's memory threshold, the system summarizes historical interactions while preserving the most recent skill instructions.¹⁴ The active skill instruction set is kept within a dedicated, re-attached token budget ¹⁴:

$$T_{\text{active}} \leq 25,000 \text{ tokens}$$

This keeps critical step-by-step instructions in the context window even during long coding sessions.¹⁴

Secure Tunnelling and P2P Networking Layer

To safely expose the local macOS development environment to a mobile device over public networks, the system implements an encrypted, zero-trust network topology using **Tailscale**.⁶



```
| (Bound to 127.0.0.1)
```

```
+-----+
```

Standalone System Extension Configuration

To bypass the sandboxing limitations and Screen Time network filter conflicts typical of the Mac App Store client, developers must use the **Standalone** variant of Tailscale for macOS.²⁹ The Standalone variant runs as a highly secure macOS system extension with root privileges, operating within an isolated system container to manage peer-to-peer WireGuard connections.²⁹

Once authenticated via a Single Sign-On (SSO) identity provider, the devices are assigned stable, non-routable 100.x.y.z IP addresses within the private "tailnet".²⁸

Zero-Exposure Port Proxying using Tailscale Serve

The local orchestrator (e.g., FastAPI running on port 8000) is bound strictly to loopback (127.0.0.1) to prevent external exposure.⁵ To share this port securely with the mobile client, the host runs **Tailscale Serve**³⁰:

```
Bash
```

```
tailscale serve 8000
```

This configuration serves several key security functions:

- **P2P WireGuard Encryption:** It intercepts incoming mobile requests on the network adapter, routes them through the encrypted P2P tunnel, and proxies them to the local port.⁶
- **Local TLS Termination:** Tailscale Serve automatically provisions and manages HTTPS certificates using the host's private MagicDNS name (e.g., <https://mac-mini.ts.net>), ensuring that transport layer security is active even when transitioning across unsecured public hotspots.²⁸
- **Spoof-Resistant Identity Verification:** Tailscale Serve automatically appends cryptographically verified HTTP headers to inbound requests, allowing the FastAPI gateway to confirm the user's identity and reject unauthorized access attempts³⁰:
 - Tailscale-User-Login: Contains the sender's authenticated email (e.g., developer@domain.com).³⁰
 - Tailscale-User-Name: Contains the sender's validated display name.³⁰

Mobile Frontend and Human-in-the-Loop Integration

The mobile application acts as a clean, responsive console for orchestrating the developer's macOS projects, built using **React Native** and **Expo**.³²

The core chat UI is built on **React Native Gifted Chat**, which handles bubble rendering, input toolbars, typing indicators, and system attachment events.³³ This keeps the interface lightweight and responsive.³³ The application connects to the macOS backend using standard WebSocket protocols securely routed within the private tailnet.⁶ State is managed on-device using a lightweight storage layer that combines **Zustand** (for active conversation states) and **react-native-mmkv** (for secure, persistent disk storage).³²

TypeScript

```
// Core configuration interface for the mobile GiftedChat screen
import React, { useState, useEffect } from 'react';
import { GiftedChat } from 'react-native-gifted-chat'; [33]

export function RemoteDeveloperConsole() {
  const [messages, setMessages] = useState(); [33]
  const = useState("");

  // Manage secure WebSocket connections over the private Tailscale tunnel
  useEffect(() => {
    const ws = new WebSocket('ws://mac-mini.ts.net/ws/orchestration');
    ws.onmessage = (event) => {
      const payload = JSON.parse(event.data);
      if (payload.type === 'telemetry') {
        setSystemUptime(`CPU: ${payload.cpu_utilization}% | RAM: ${payload.ram_used_gb} GB`);
      } else if (payload.type === 'chat_response') {
        setMessages((prev) => GiftedChat.append(prev, [payload.message])); [33]
      }
    };
    return () => ws.close();
  },);

  return (
    <GiftedChat
      messages={messages} [33]
      user={{ _id: 1, name: "Lead Developer" }} [33]
```

```

    onSend={(newMsgs) => {
      // Broadcast user prompts and workspace commands securely
    }}
  />
);
}

```

LangGraph Human-in-the-Loop Execution Gate

To prevent the local agent from executing destructive commands (such as unauthorized file overwrites, forced repository changes, or system reboots) autonomously, the orchestrator implements a strict, state-gated execution loop.¹⁵ In regulated or critical workflows, fully autonomous agents present too high an operational risk.³⁵ This system utilizes LangGraph's native interrupt and Command pattern to pause execution and request user approval for sensitive actions.³⁵

Python

```

from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

# Define the shared memory state of the compilation graph
class CompilationState(dict):
    active_directory: str
    proposed_command: str
    is_approved: bool
    console_logs: str

def request_approval_node(state: CompilationState):
    # Pause graph execution dynamically using LangGraph's native interrupt
    approval_response = interrupt({
        "alert": "Awaiting Command Verification",
        "command": state["proposed_command"],
        "working_directory": state["active_directory"]
    })

    return {
        "is_approved": approval_response.get("approved", False)
    }

def execute_command_node(state: CompilationState):

```

```

if state["is_approved"]:
    # Execute the verified compilation or deployment script
    logs = "Compilation Succeeded. Port 3000 online."
    return {"console_logs": logs}
else:
    return {"console_logs": "Execution terminated: Command rejected."}

# Build the state graph with a persistent checkpoint ledger
builder = StateGraph(CompilationState)
builder.add_node("request_approval_node", request_approval_node)
builder.add_node("execute_command_node", execute_command_node)

builder.add_edge(START, "request_approval_node")
builder.add_edge("request_approval_node", "execute_command_node")
builder.add_edge("execute_command_node", END)

# Compile the graph with a memory checkpoint for state persistence and replayability
memory_ledger = SqliteSaver.from_conn_string(":memory:")
compiled_orchestrator_graph = builder.compile(checkpointer=memory_ledger)

```

This pattern ensures that:

- **Deterministic Pauses:** The graph automatically pauses execution whenever a sensitive system command is generated.³⁵
- **State Preservation:** The orchestrator serializes the current state and preserves it within a SQLite checkpoint store.³⁵
- **Human Verification:** The mobile interface prompts the developer with the exact proposed shell command.³⁷ Once verified, the developer submits an approval signal via WebSocket.³⁶
- **Resume Capability:** The orchestrator resumes execution using the `Command(resume=payload)` method, allowing developers to execute tasks safely on the go.³⁵

Technical Deployment Checklist

Implementing this system requires configuring the local macOS host, the network layer, and the mobile client step-by-step.

1. Host Machine Setup

For general hardware configurations, execute the standard Ollama installation script ⁵:

Bash

```
curl -fsSL https://ollama.com/install.sh | sh
```

To ensure Ollama accepts remote requests strictly from the secure Tailscale network, set the system environment variables ³⁸:

Bash

```
launchctl setenv OLLAMA_HOST "127.0.0.1:11434"
```

Restart the Ollama application to apply the changes.³⁸ Next, download a high-performance local coding model ⁵:

Bash

```
ollama pull qwen3
```

For Apple Silicon Macs, developers can alternatively set up an **MLX-LM** server for faster local inference ¹¹:

- Install Python 3.12 via Homebrew ¹²:

Bash

```
brew install python@3.12
```

- Create a dedicated virtual environment and install MLX-LM ¹²:

Bash

```
/opt/homebrew/bin/python3.12 -m venv ~/mlx-env
```

```
source ~/mlx-env/bin/activate
```

```
pip install --upgrade pip
```

```
pip install mlx-lm
```

- Start the OpenAI-compatible model server, loading an optimized Qwen Coder model ¹²:

Bash

```
mlx_lm.server --model mlx-community/Qwen3-Coder-30B-A3B-Instruct-8bit --port
```

8080

2. Tailscale Network Setup

- Download and install the Standalone variant of Tailscale on the macOS host.²⁹
- Install the Tailscale app on the target mobile device, logging in with the same SSO identity provider to join the tailnet.²⁸
- Enable **VPN On-Demand** in the mobile Tailscale client settings so the device automatically connects to the tailnet when exiting local Wi-Fi coverage.³¹
- Enable MagicDNS in the Tailscale admin console to resolve device names automatically.⁶

3. macOS Orchestrator Setup

- Create the local python orchestrator workspace:
Bash
`mkdir -p ~/developer_orchestrator/skills`
`cd ~/developer_orchestrator`
- Define a standard python dependency file (requirements.txt) containing the runtime dependencies:
fastapi==0.110.0
uvicorn==0.28.0
langgraph==0.0.26
psutil==5.9.8
ollama==0.1.7
- Install the dependencies using the virtual environment:
Bash
`pip install -r requirements.txt`
- Implement the FastAPI server endpoints to manage connection events, parse active workspace metrics, fetch system parameters, and stream agent responses back to the mobile client.²¹

4. Mobile Client Application Bootstrapping

- Initialize a new React Native workspace using the Expo command-line utility⁴⁰:
Bash
`npx create-expo-app MobileOrchestratorApp --template blank`
`cd MobileOrchestratorApp`
- Install the essential navigation, visual UI, and networking dependencies³³:
Bash
`npx expo install react-native-gifted-chat react-native-reanimated`

```
react-native-gesture-handler react-native-safe-area-context  
react-native-keyboard-controller
```

- Configure the development client configuration profiles and launch the local bundler ⁴⁰:

```
Bash
```

```
npx expo start --dev-client
```

Resource Allocation and Scaling Equations

To maintain high throughput and prevent system latency when running a local 7B-9B parameter model alongside development processes on the macOS host, resource allocation is managed programmatically.⁵ Memory allocation scales based on active model parameters, context window lengths, and parallel request limits.³⁸

The system calculates memory requirements using the following scaling model:

$$M_{\text{total}} = S_{\text{model}} \cdot Q_{\text{factor}} + (N_{\text{parallel}} \cdot L_{\text{context}} \cdot K_{\text{bytes}})$$

where:

- M_{total} represents the total system unified memory requirement in gigabytes.³⁸
- S_{model} is the raw parameter count of the model (e.g., 8×10^9 parameters for Llama-3.1-8B).⁵
- Q_{factor} is the quantization factor (typically 0.5 bytes for 4-bit precision, or 1.0 bytes for 8-bit precision).¹¹
- N_{parallel} is the maximum number of concurrent requests processed by the model.³⁸
- L_{context} is the maximum context length in tokens.³⁸
- K_{bytes} is the model-specific Key-Value (KV) cache memory cost per token (typically 2×10^{-7} GB/token for standard architectures).

To avoid host system constraints, Ollama manages concurrent requests using the following configuration values ³⁸:

`OLLAMA_NUM_PARALLEL = 2` and `OLLAMA_MAX`

Limiting concurrent threads prevents memory thrashing and maintains fast execution speeds, keeping response times low on the mobile client.⁵ When the workstation faces high compilation loads, the orchestrator throttles background agent tasks, prioritizing CPU cycles for core build operations and test execution.²⁰

Works cited

1. Claude Code | Anthropic's agentic coding system, accessed May 19, 2026, <https://www.anthropic.com/product/claude-code>
2. Google Antigravity 2.0 is Your New Android Coding Hub; Google AI Studio Gets Mobile App, accessed May 19, 2026, <https://www.androidheadlines.com/2026/05/google-io-2026-antigravity-developer-tools-ai-studio-mobile-app.html>
3. Overview - Claude Code Docs, accessed May 19, 2026, <https://code.claude.com/docs/en/overview>
4. Google flips Antigravity into an agentic dev suite, AI Studio app lands on Android, accessed May 19, 2026, <https://9to5google.com/2026/05/19/google-antigravity-agentic-developer-suite/>
5. Run Mistral LLM Locally on macOS with Ollama: From Zero to Working API - Medium, accessed May 19, 2026, <https://medium.com/@sachin2713/run-mistral-llm-locally-on-macos-with-ollama-from-zero-to-working-api-f5c76e6b8060>
6. Connect to devices · TailScale Docs, accessed May 19, 2026, <https://tailscale.com/docs/how-to/connect-to-devices>
7. Dynamic Agent is a Rust-based framework for building Retrieval-Augmented Generation (RAG) AI agents with multi-LLM and multi-vector store support, featuring WebSocket communication, caching, and flexible configuration. · GitHub, accessed May 19, 2026, <https://github.com/DevsHero/dynamic-agent>
8. openinterpreter/open-interpreter: A natural language interface for computers - GitHub, accessed May 19, 2026, <https://github.com/openinterpreter/open-interpreter>
9. 2FastLabs/agent-squad: Flexible and powerful framework for managing multiple AI agents and handling complex conversations - GitHub, accessed May 19, 2026, <https://github.com/2FastLabs/agent-squad>
10. Open Interpreter download | SourceForge.net, accessed May 19, 2026, <https://sourceforge.net/projects/open-interpreter.mirror/>
11. ml-explore/mlx-lm: Run LLMs with MLX - GitHub, accessed May 19, 2026, <https://github.com/ml-explore/mlx-lm>
12. Running Local LLMs as Your AI Coding Assistant on Apple Silicon - DEV Community, accessed May 19, 2026, <https://dev.to/brunocerberus/running-local-llms-on-apple-silicon-2ecm>
13. The Complete Guide to Ollama: Run Large Language Models Locally - DEV Community, accessed May 19, 2026, <https://dev.to/ajitkumar/the-complete-guide-to-ollama-run-large-language-models-locally-2mge>

14. Extend Claude with skills - Claude Code Docs, accessed May 19, 2026, <https://code.claude.com/docs/en/skills>
15. How Claude Code works - Claude Code Docs, accessed May 19, 2026, <https://code.claude.com/docs/en/how-claude-code-works>
16. An important update: Transitioning Gemini CLI to Antigravity CLI ..., accessed May 19, 2026, <https://developers.googleblog.com/an-important-update-transitioning-gemini-cli-to-antigravity-cli/>
17. Claude Code CLI: Command-Line AI Coding for Real Developer Workflows - DataCamp, accessed May 19, 2026, <https://www.datacamp.com/de/tutorial/claude-code-cli>
18. michaelw9999/antigravity-cli: Unofficial CLI tools to view ... - GitHub, accessed May 19, 2026, <https://github.com/michaelw9999/antigravity-cli>
19. I/O 2026 developer highlights: Antigravity, Gemini API, AI Studio - Google Blog, accessed May 19, 2026, <https://blog.google/innovation-and-ai/technology/developers-tools/google-io-2026-developer-highlights/>
20. Home - psutil documentation, accessed May 19, 2026, <https://psutil.readthedocs.io/latest/index.html>
21. System Monitoring Made Easy with Python's Psutil Library | by Ravi Tiwari | Medium, accessed May 19, 2026, <https://umeey.medium.com/system-monitoring-made-easy-with-pythons-psutil-library-4b9add95a443>
22. How to Get the list of Running Processes using Python? - TutorialsPoint, accessed May 19, 2026, <https://www.tutorialspoint.com/article/how-to-get-the-list-of-running-processes-using-python>
23. Run Claude Code programmatically - Claude Code Docs, accessed May 19, 2026, <https://code.claude.com/docs/en/headless>
24. CLI reference - Claude Code Docs, accessed May 19, 2026, <https://code.claude.com/docs/en/cli-reference>
25. Agent Skills - Claude API Docs, accessed May 19, 2026, <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
26. Claude Skills, Explained: A Developer's Guide to Building Repeatable AI Tasks - Medium, accessed May 19, 2026, <https://medium.com/@yuxiaojian/claude-skills-explained-a-developers-guide-to-building-repeatable-ai-tasks-ddee3beec475>
27. Best Claude Code Skills to Try in 2026 - Firecrawl, accessed May 19, 2026, <https://www.firecrawl.dev/blog/best-claude-code-skills>
28. Tailscale quickstart, accessed May 19, 2026, <https://tailscale.com/docs/how-to/quickstart>
29. Three ways to run Tailscale on macOS, accessed May 19, 2026, <https://tailscale.com/docs/concepts/macos-variants>
30. Tailscale Serve · Tailscale Docs, accessed May 19, 2026, <https://tailscale.com/docs/features/tailscale-serve>

31. Using VPN On Demand for iOS and macOS · Tailscale Docs, accessed May 19, 2026, <https://tailscale.com/docs/features/client/ios-vpn-on-demand>
32. React Native Template by obytes - GitHub, accessed May 19, 2026, <https://github.com/obytes/react-native-template-obytes>
33. FaridSafi/react-native-gifted-chat - GitHub, accessed May 19, 2026, <https://github.com/faridsafi/react-native-gifted-chat>
34. Production AI Agent Architecture with LangGraph & FastAPI - LangQuang, accessed May 19, 2026, <https://langquang.com/blogs/ai-agent-architecture-langgraph-fastapi-for-productionready-chatbots>
35. Human-in-the-Loop AI: Time-Travel Workflows with LangGraph - Christian Mendieta, accessed May 19, 2026, <https://christianmendieta.ca/human-in-the-loop-ai-time-travel-workflows-with-langgraph/>
36. Human-in-the-loop approval dashboard for LangGraph agents — open source, free to deploy - LangChain Forum, accessed May 19, 2026, <https://forum.langchain.com/t/human-in-the-loop-approval-dashboard-for-langgraph-agents-open-source-free-to-deploy/3616>
37. LangGraph Uncovered: AI Agent and Human-in-the-Loop: Enhancing Decision-Making with Intelligent Automation Part -III - DEV Community, accessed May 19, 2026, <https://dev.to/sreeni5018/langgraph-uncoveredai-agent-and-human-in-the-loop-enhancing-decision-making-with-intelligent-3dbc>
38. FAQ - Ollama's documentation, accessed May 19, 2026, <https://docs.ollama.com/faq>
39. Simple Chat App With React Native - Part II - Callstack, accessed May 19, 2026, <https://www.callstack.com/blog/simple-chat-app-with-react-native-part-2-front-end>
40. React Native Chat App Expo Tutorial - GetStream.io, accessed May 19, 2026, <https://getstream.io/chat/sdk/react-native/tutorial/expo/>
41. Best Local Model for Openclaw : r/LocalLLaMA - Reddit, accessed May 19, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1qtnz9s/best_local_model_for_openclaw/